EXPERIMENT

Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

Name **:** S.Nagulan

Reg No **:** 1924260216

(This experiment is divided into 5 problems)

Problem 1: Play Tennis Data Set

Aim

To write a Python program that implements the Naïve Bayes algorithm on the Play Tennis Data Set to decide whether tennis can be played under given weather conditions, and to display the results using a confusion matrix and the accuracy.

Procedure

1. Store the training examples (4 features: Outlook, Temperature, Humidity, Wind; class: PlayTennis) and the test examples as lists in Python.
2. Training: count the examples of each class to find the prior probability P(C), and count how often each feature value occurs with each class to find the likelihood P(x | C).
3. Apply Laplace smoothing, P(x | C) = (count + 1) / (class count + number of values of the feature), so that no probability becomes zero.
4. Prediction: for every test example calculate the posterior value P(C) x P(x1 | C) x P(x2 | C) x ... for each class and choose the class with the maximum value.
5. Compare the predicted class with the actual class of every test example and build the confusion matrix (rows = actual class, columns = predicted class).
6. Calculate the accuracy = (sum of the diagonal of the confusion matrix / total test examples) x 100 and display the results.

Coding (Python)

```
# Naive Bayes Classifier - Play Tennis Data Set
from collections import Counter

features = ['Outlook', 'Temperature', 'Humidity', 'Wind']

# Training data: [Outlook, Temperature, Humidity, Wind, Class]
train = [
    ['Sunny', 'Hot', 'High', 'Weak', 'No'],
    ['Sunny', 'Hot', 'High', 'Strong', 'No'],
    ['Overcast', 'Hot', 'High', 'Weak', 'Yes'],
    ['Rainy', 'Mild', 'High', 'Weak', 'Yes'],
    ['Rainy', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rainy', 'Cool', 'Normal', 'Strong', 'No'],
    ['Overcast', 'Cool', 'Normal', 'Strong', 'Yes'],
    ['Sunny', 'Mild', 'High', 'Weak', 'No'],
    ['Sunny', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rainy', 'Mild', 'Normal', 'Weak', 'Yes'],
    ['Sunny', 'Mild', 'Normal', 'Strong', 'Yes'],
    ['Overcast', 'Mild', 'High', 'Strong', 'Yes'],
    ['Overcast', 'Hot', 'Normal', 'Weak', 'Yes'],
    ['Rainy', 'Mild', 'High', 'Strong', 'No'],
]

# Test data: [Outlook, Temperature, Humidity, Wind, Actual class]
test = [
    ['Sunny', 'Cool', 'High', 'Strong', 'No'],
    ['Overcast', 'Mild', 'Normal', 'Weak', 'Yes'],
    ['Rainy', 'Hot', 'High', 'Strong', 'No'],
```

```
    ['Sunny', 'Hot', 'Normal', 'Weak', 'Yes'],
    ['Rainy', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Sunny', 'Mild', 'High', 'Strong', 'No'],
]

def train_naive_bayes(train):
    labels = [r[-1] for r in train]
    class_count = Counter(labels)
    n_features = len(train[0]) - 1
    # values[i] = set of all values taken by feature i
    values = [set(r[i] for r in train) for i in range(n_features)]
    # count[(class, i, value)] = number of training rows
    count = Counter()
    for r in train:
        for i in range(n_features):
            count[(r[-1], i, r[i])] += 1
    return class_count, values, count

def predict(model, x, n_rows):
    class_count, values, count = model
    best_class, best_prob = None, -1
    for c in class_count:
        prob = class_count[c] / n_rows                    # prior P(c)
        for i, v in enumerate(x):
            # likelihood P(v | c) with Laplace smoothing
            prob *= (count[(c, i, v)] + 1) / (class_count[c] + len(values[i]))
        if prob > best_prob:
            best_class, best_prob = c, prob
    return best_class

model = train_naive_bayes(train)
classes = sorted(model[0])

actual, predicted = [], []
print("Test sample | Actual | Predicted")
for row in test:
    p = predict(model, row[:-1], len(train))
    actual.append(row[-1])
    predicted.append(p)
    print(row[:-1], "|", row[-1], "|", p)

# Confusion matrix (rows = actual class, columns = predicted class)
matrix = [[0] * len(classes) for _ in classes]
for a, p in zip(actual, predicted):
    matrix[classes.index(a)][classes.index(p)] += 1

print("\nConfusion Matrix")
print("Class order (rows = actual, columns = predicted):", classes)
for c, row in zip(classes, matrix):
    print("Actual", c, ":", row)

correct = sum(matrix[i][i] for i in range(len(classes)))
accuracy = correct / len(test) * 100
print("\nCorrect predictions :", correct, "out of", len(test))
print("Accuracy :", round(accuracy, 2), "%")
```

Input

Training data set (14 examples):

Outlook	Temperature	Humidity	Wind	PlayTennis
Sunny	Hot	High	Weak	No

Outlook	Temperature	Humidity	Wind	PlayTennis
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rainy	Mild	High	Weak	Yes
Rainy	Cool	Normal	Weak	Yes
Rainy	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rainy	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rainy	Mild	High	Strong	No

Test data set (6 examples):

Outlook	Temperature	Humidity	Wind	PlayTennis
Sunny	Cool	High	Strong	No
Overcast	Mild	Normal	Weak	Yes
Rainy	Hot	High	Strong	No
Sunny	Hot	Normal	Weak	Yes
Rainy	Cool	Normal	Weak	Yes
Sunny	Mild	High	Strong	No

Output

```
Test sample | Actual | Predicted
['Sunny', 'Cool', 'High', 'Strong'] | No | No
['Overcast', 'Mild', 'Normal', 'Weak'] | Yes | Yes
['Rainy', 'Hot', 'High', 'Strong'] | No | No
['Sunny', 'Hot', 'Normal', 'Weak'] | Yes | Yes
['Rainy', 'Cool', 'Normal', 'Weak'] | Yes | Yes
['Sunny', 'Mild', 'High', 'Strong'] | No | No

Confusion Matrix
Class order (rows = actual, columns = predicted): ['No', 'Yes']
Actual No : [3, 0]
Actual Yes : [0, 3]

Correct predictions : 6 out of 6
Accuracy : 100.0 %
```

Result

Thus the Naïve Bayes algorithm was implemented in Python successfully. The classifier predicted **6 out of 6** test examples correctly, as shown in the confusion matrix, and achieved an accuracy of **100.0 %**.

Problem 2: Email Spam Data Set

Aim

To write a Python program that implements the Naïve Bayes algorithm on the Email Spam Data Set to decide whether an e-mail is spam, and to display the results using a confusion matrix and the accuracy.

Procedure

1. Store the training examples (4 features: Sender, HasLink, Attachment, Keyword; class: Spam) and the test examples as lists in Python.
2. Training: count the examples of each class to find the prior probability P(C), and count how often each feature value occurs with each class to find the likelihood P(x | C).
3. Apply Laplace smoothing, P(x | C) = (count + 1) / (class count + number of values of the feature), so that no probability becomes zero.
4. Prediction: for every test example calculate the posterior value P(C) x P(x1 | C) x P(x2 | C) x ... for each class and choose the class with the maximum value.
5. Compare the predicted class with the actual class of every test example and build the confusion matrix (rows = actual class, columns = predicted class).
6. Calculate the accuracy = (sum of the diagonal of the confusion matrix / total test examples) x 100 and display the results.

Coding (Python)

```
# Naive Bayes Classifier - Email Spam Data Set
from collections import Counter

features = ['Sender', 'HasLink', 'Attachment', 'Keyword']

# Training data: [Sender, HasLink, Attachment, Keyword, Class]
train = [
    ['Unknown', 'Yes', 'Yes', 'Offer', 'Yes'],
    ['Unknown', 'Yes', 'No', 'Offer', 'Yes'],
    ['Known', 'No', 'No', 'Meeting', 'No'],
    ['Known', 'Yes', 'No', 'Meeting', 'No'],
    ['Unknown', 'No', 'No', 'Offer', 'Yes'],
    ['Known', 'No', 'Yes', 'Offer', 'No'],
    ['Unknown', 'No', 'Yes', 'Meeting', 'No'],
    ['Unknown', 'Yes', 'No', 'Meeting', 'No'],
    ['Known', 'Yes', 'Yes', 'Offer', 'No'],
    ['Unknown', 'Yes', 'Yes', 'Offer', 'Yes'],
]

# Test data: [Sender, HasLink, Attachment, Keyword, Actual class]
test = [
    ['Unknown', 'No', 'No', 'Offer', 'Yes'],
    ['Known', 'Yes', 'No', 'Meeting', 'No'],
    ['Unknown', 'Yes', 'No', 'Meeting', 'No'],
    ['Known', 'No', 'No', 'Offer', 'No'],
    ['Unknown', 'Yes', 'Yes', 'Meeting', 'Yes'],
]
def train_naive_bayes(train):
```

```
    labels = [r[-1] for r in train]
    class_count = Counter(labels)
    n_features = len(train[0]) - 1
    # values[i] = set of all values taken by feature i
    values = [set(r[i] for r in train) for i in range(n_features)]
    # count[(class, i, value)] = number of training rows
    count = Counter()
    for r in train:
        for i in range(n_features):
            count[(r[-1], i, r[i])] += 1
    return class_count, values, count

def predict(model, x, n_rows):
    class_count, values, count = model
    best_class, best_prob = None, -1
    for c in class_count:
        prob = class_count[c] / n_rows                    # prior P(c)
        for i, v in enumerate(x):
            # likelihood P(v | c) with Laplace smoothing
            prob *= (count[(c, i, v)] + 1) / (class_count[c] + len(values[i]))
        if prob > best_prob:
            best_class, best_prob = c, prob
    return best_class

model = train_naive_bayes(train)
classes = sorted(model[0])

actual, predicted = [], []
print("Test sample | Actual | Predicted")
for row in test:
    p = predict(model, row[:-1], len(train))
    actual.append(row[-1])
    predicted.append(p)
    print(row[:-1], "|", row[-1], "|", p)

# Confusion matrix (rows = actual class, columns = predicted class)
matrix = [[0] * len(classes) for _ in classes]
for a, p in zip(actual, predicted):
    matrix[classes.index(a)][classes.index(p)] += 1

print("\nConfusion Matrix")
print("Class order (rows = actual, columns = predicted):", classes)
for c, row in zip(classes, matrix):
    print("Actual", c, ":", row)

correct = sum(matrix[i][i] for i in range(len(classes)))
accuracy = correct / len(test) * 100
print("\nCorrect predictions :", correct, "out of", len(test))
print("Accuracy :", round(accuracy, 2), "%")
```

Input

Training data set (10 examples):

Sender	HasLink	Attachment	Keyword	Spam
Unknown	Yes	Yes	Offer	Yes
Unknown	Yes	No	Offer	Yes
Known	No	No	Meeting	No
Known	Yes	No	Meeting	No
Unknown	No	No	Offer	Yes

Sender	HasLink	Attachment	Keyword	Spam
Known	No	Yes	Offer	No
Unknown	No	Yes	Meeting	No
Unknown	Yes	No	Meeting	No
Known	Yes	Yes	Offer	No
Unknown	Yes	Yes	Offer	Yes

Test data set (5 examples):

Sender	HasLink	Attachment	Keyword	Spam
Unknown	No	No	Offer	Yes
Known	Yes	No	Meeting	No
Unknown	Yes	No	Meeting	No
Known	No	No	Offer	No
Unknown	Yes	Yes	Meeting	Yes

Output

```
Test sample | Actual | Predicted
['Unknown', 'No', 'No', 'Offer'] | Yes | Yes
['Known', 'Yes', 'No', 'Meeting'] | No | No
['Unknown', 'Yes', 'No', 'Meeting'] | No | No
['Known', 'No', 'No', 'Offer'] | No | No
['Unknown', 'Yes', 'Yes', 'Meeting'] | Yes | No

Confusion Matrix
Class order (rows = actual, columns = predicted): ['No', 'Yes']
Actual No : [3, 0]
Actual Yes : [1, 1]

Correct predictions : 4 out of 5
Accuracy : 80.0 %
```

Result

Thus the Naïve Bayes algorithm was implemented in Python successfully. The classifier predicted **4 out of 5** test examples correctly, as shown in the confusion matrix, and achieved an accuracy of **80.0 %**.

Problem 3: Customer Buys Computer Data Set

Aim

To write a Python program that implements the Naïve Bayes algorithm on the Customer Buys Computer Data Set to decide whether a customer will buy a computer, and to display the results using a confusion matrix and the accuracy.

Procedure

1. Store the training examples (4 features: Age, Income, Student, CreditRating; class: BuysComputer) and the test examples as lists in Python.
2. Training: count the examples of each class to find the prior probability P(C), and count how often each feature value occurs with each class to find the likelihood P(x | C).
3. Apply Laplace smoothing, P(x | C) = (count + 1) / (class count + number of values of the feature), so that no probability becomes zero.
4. Prediction: for every test example calculate the posterior value P(C) x P(x1 | C) x P(x2 | C) x ... for each class and choose the class with the maximum value.
5. Compare the predicted class with the actual class of every test example and build the confusion matrix (rows = actual class, columns = predicted class).
6. Calculate the accuracy = (sum of the diagonal of the confusion matrix / total test examples) x 100 and display the results.

Coding (Python)

```
# Naive Bayes Classifier - Customer Buys Computer Data Set
from collections import Counter

features = ['Age', 'Income', 'Student', 'CreditRating']

# Training data: [Age, Income, Student, CreditRating, Class]
train = [
    ['Youth', 'High', 'No', 'Fair', 'No'],
    ['Youth', 'High', 'No', 'Excellent', 'No'],
    ['Middle', 'High', 'No', 'Fair', 'Yes'],
    ['Senior', 'Medium', 'No', 'Fair', 'Yes'],
    ['Senior', 'Low', 'Yes', 'Fair', 'Yes'],
    ['Senior', 'Low', 'Yes', 'Excellent', 'No'],
    ['Middle', 'Low', 'Yes', 'Excellent', 'Yes'],
    ['Youth', 'Medium', 'No', 'Fair', 'No'],
    ['Youth', 'Low', 'Yes', 'Fair', 'Yes'],
    ['Senior', 'Medium', 'Yes', 'Fair', 'Yes'],
    ['Youth', 'Medium', 'Yes', 'Excellent', 'Yes'],
    ['Middle', 'Medium', 'No', 'Excellent', 'Yes'],
    ['Middle', 'High', 'Yes', 'Fair', 'Yes'],
    ['Senior', 'Medium', 'No', 'Excellent', 'No'],
]

# Test data: [Age, Income, Student, CreditRating, Actual class]
test = [
    ['Youth', 'Medium', 'Yes', 'Fair', 'Yes'],
    ['Senior', 'Low', 'No', 'Excellent', 'No'],
    ['Middle', 'Low', 'No', 'Fair', 'Yes'],
```

```
    ['Youth', 'High', 'No', 'Fair', 'No'],
    ['Senior', 'High', 'Yes', 'Excellent', 'No'],
    ['Youth', 'Low', 'Yes', 'Excellent', 'Yes'],
]

def train_naive_bayes(train):
    labels = [r[-1] for r in train]
    class_count = Counter(labels)
    n_features = len(train[0]) - 1
    # values[i] = set of all values taken by feature i
    values = [set(r[i] for r in train) for i in range(n_features)]
    # count[(class, i, value)] = number of training rows
    count = Counter()
    for r in train:
        for i in range(n_features):
            count[(r[-1], i, r[i])] += 1
    return class_count, values, count

def predict(model, x, n_rows):
    class_count, values, count = model
    best_class, best_prob = None, -1
    for c in class_count:
        prob = class_count[c] / n_rows                    # prior P(c)
        for i, v in enumerate(x):
            # likelihood P(v | c) with Laplace smoothing
            prob *= (count[(c, i, v)] + 1) / (class_count[c] + len(values[i]))
        if prob > best_prob:
            best_class, best_prob = c, prob
    return best_class

model = train_naive_bayes(train)
classes = sorted(model[0])

actual, predicted = [], []
print("Test sample | Actual | Predicted")
for row in test:
    p = predict(model, row[:-1], len(train))
    actual.append(row[-1])
    predicted.append(p)
    print(row[:-1], "|", row[-1], "|", p)

# Confusion matrix (rows = actual class, columns = predicted class)
matrix = [[0] * len(classes) for _ in classes]
for a, p in zip(actual, predicted):
    matrix[classes.index(a)][classes.index(p)] += 1

print("\nConfusion Matrix")
print("Class order (rows = actual, columns = predicted):", classes)
for c, row in zip(classes, matrix):
    print("Actual", c, ":", row)

correct = sum(matrix[i][i] for i in range(len(classes)))
accuracy = correct / len(test) * 100
print("\nCorrect predictions :", correct, "out of", len(test))
print("Accuracy :", round(accuracy, 2), "%")
```

Input

Training data set (14 examples):

Age	Income	Student	CreditRating	BuysComputer
Youth	High	No	Fair	No

Age	Income	Student	CreditRating	BuysComputer
Youth	High	No	Excellent	No
Middle	High	No	Fair	Yes
Senior	Medium	No	Fair	Yes
Senior	Low	Yes	Fair	Yes
Senior	Low	Yes	Excellent	No
Middle	Low	Yes	Excellent	Yes
Youth	Medium	No	Fair	No
Youth	Low	Yes	Fair	Yes
Senior	Medium	Yes	Fair	Yes
Youth	Medium	Yes	Excellent	Yes
Middle	Medium	No	Excellent	Yes
Middle	High	Yes	Fair	Yes
Senior	Medium	No	Excellent	No

Test data set (6 examples):

Age	Income	Student	CreditRating	BuysComputer
Youth	Medium	Yes	Fair	Yes
Senior	Low	No	Excellent	No
Middle	Low	No	Fair	Yes
Youth	High	No	Fair	No
Senior	High	Yes	Excellent	No
Youth	Low	Yes	Excellent	Yes

Output

```
Test sample | Actual | Predicted
['Youth', 'Medium', 'Yes', 'Fair'] | Yes | Yes
['Senior', 'Low', 'No', 'Excellent'] | No | No
['Middle', 'Low', 'No', 'Fair'] | Yes | Yes
['Youth', 'High', 'No', 'Fair'] | No | No
['Senior', 'High', 'Yes', 'Excellent'] | No | Yes
['Youth', 'Low', 'Yes', 'Excellent'] | Yes | Yes

Confusion Matrix
Class order (rows = actual, columns = predicted): ['No', 'Yes']
Actual No : [2, 1]
Actual Yes : [0, 3]

Correct predictions : 5 out of 6
Accuracy : 83.33 %
```

Result

Thus the Naïve Bayes algorithm was implemented in Python successfully. The classifier predicted **5 out of 6** test examples correctly, as shown in the confusion matrix, and achieved an accuracy of **83.33 %**.

Problem 4: Loan Approval Data Set

Aim

To write a Python program that implements the Naïve Bayes algorithm on the Loan Approval Data Set to decide whether a bank loan application is approved, and to display the results using a confusion matrix and the accuracy.

Procedure

1. Store the training examples (4 features: Income, Credit, Employment, Property; class: Approved) and the test examples as lists in Python.
2. Training: count the examples of each class to find the prior probability P(C), and count how often each feature value occurs with each class to find the likelihood P(x | C).
3. Apply Laplace smoothing, P(x | C) = (count + 1) / (class count + number of values of the feature), so that no probability becomes zero.
4. Prediction: for every test example calculate the posterior value P(C) x P(x1 | C) x P(x2 | C) x ... for each class and choose the class with the maximum value.
5. Compare the predicted class with the actual class of every test example and build the confusion matrix (rows = actual class, columns = predicted class).
6. Calculate the accuracy = (sum of the diagonal of the confusion matrix / total test examples) x 100 and display the results.

Coding (Python)

```
# Naive Bayes Classifier - Loan Approval Data Set
from collections import Counter

features = ['Income', 'Credit', 'Employment', 'Property']

# Training data: [Income, Credit, Employment, Property, Class]
train = [
    ['High', 'Good', 'Salaried', 'Owned', 'Yes'],
    ['Low', 'Poor', 'Unemployed', 'Rented', 'No'],
    ['High', 'Good', 'Self-Employed', 'Owned', 'Yes'],
    ['Low', 'Good', 'Salaried', 'Owned', 'Yes'],
    ['High', 'Poor', 'Salaried', 'Rented', 'No'],
    ['Low', 'Poor', 'Salaried', 'Owned', 'No'],
    ['High', 'Good', 'Salaried', 'Rented', 'Yes'],
    ['Low', 'Good', 'Unemployed', 'Rented', 'No'],
    ['High', 'Poor', 'Self-Employed', 'Owned', 'No'],
    ['Low', 'Good', 'Self-Employed', 'Owned', 'Yes'],
]

# Test data: [Income, Credit, Employment, Property, Actual class]
test = [
    ['High', 'Good', 'Unemployed', 'Owned', 'Yes'],
    ['Low', 'Poor', 'Self-Employed', 'Rented', 'No'],
    ['High', 'Poor', 'Salaried', 'Owned', 'No'],
    ['Low', 'Good', 'Salaried', 'Rented', 'Yes'],
    ['Low', 'Good', 'Unemployed', 'Owned', 'No'],
]
def train_naive_bayes(train):
```

```
    labels = [r[-1] for r in train]
    class_count = Counter(labels)
    n_features = len(train[0]) - 1
    # values[i] = set of all values taken by feature i
    values = [set(r[i] for r in train) for i in range(n_features)]
    # count[(class, i, value)] = number of training rows
    count = Counter()
    for r in train:
        for i in range(n_features):
            count[(r[-1], i, r[i])] += 1
    return class_count, values, count

def predict(model, x, n_rows):
    class_count, values, count = model
    best_class, best_prob = None, -1
    for c in class_count:
        prob = class_count[c] / n_rows                    # prior P(c)
        for i, v in enumerate(x):
            # likelihood P(v | c) with Laplace smoothing
            prob *= (count[(c, i, v)] + 1) / (class_count[c] + len(values[i]))
        if prob > best_prob:
            best_class, best_prob = c, prob
    return best_class

model = train_naive_bayes(train)
classes = sorted(model[0])

actual, predicted = [], []
print("Test sample | Actual | Predicted")
for row in test:
    p = predict(model, row[:-1], len(train))
    actual.append(row[-1])
    predicted.append(p)
    print(row[:-1], "|", row[-1], "|", p)

# Confusion matrix (rows = actual class, columns = predicted class)
matrix = [[0] * len(classes) for _ in classes]
for a, p in zip(actual, predicted):
    matrix[classes.index(a)][classes.index(p)] += 1

print("\nConfusion Matrix")
print("Class order (rows = actual, columns = predicted):", classes)
for c, row in zip(classes, matrix):
    print("Actual", c, ":", row)

correct = sum(matrix[i][i] for i in range(len(classes)))
accuracy = correct / len(test) * 100
print("\nCorrect predictions :", correct, "out of", len(test))
print("Accuracy :", round(accuracy, 2), "%")
```

Input

Training data set (10 examples):

Income	Credit	Employment	Property	Approved
High	Good	Salaried	Owned	Yes
Low	Poor	Unemployed	Rented	No
High	Good	Self-Employed	Owned	Yes
Low	Good	Salaried	Owned	Yes
High	Poor	Salaried	Rented	No

Income	Credit	Employment	Property	Approved
Low	Poor	Salaried	Owned	No
High	Good	Salaried	Rented	Yes
Low	Good	Unemployed	Rented	No
High	Poor	Self-Employed	Owned	No
Low	Good	Self-Employed	Owned	Yes

Test data set (5 examples):

Income	Credit	Employment	Property	Approved
High	Good	Unemployed	Owned	Yes
Low	Poor	Self-Employed	Rented	No
High	Poor	Salaried	Owned	No
Low	Good	Salaried	Rented	Yes
Low	Good	Unemployed	Owned	No

Output

```
Test sample | Actual | Predicted
['High', 'Good', 'Unemployed', 'Owned'] | Yes | Yes
['Low', 'Poor', 'Self-Employed', 'Rented'] | No | No
['High', 'Poor', 'Salaried', 'Owned'] | No | No
['Low', 'Good', 'Salaried', 'Rented'] | Yes | Yes
['Low', 'Good', 'Unemployed', 'Owned'] | No | Yes

Confusion Matrix
Class order (rows = actual, columns = predicted): ['No', 'Yes']
Actual No : [2, 1]
Actual Yes : [0, 2]

Correct predictions : 4 out of 5
Accuracy : 80.0 %
```

Result

Thus the Naïve Bayes algorithm was implemented in Python successfully. The classifier predicted **4 out of 5** test examples correctly, as shown in the confusion matrix, and achieved an accuracy of **80.0 %**.

Problem 5: Flu Diagnosis Data Set

Aim

To write a Python program that implements the Naïve Bayes algorithm on the Flu Diagnosis Data Set to decide whether a patient has flu from the symptoms, and to display the results using a confusion matrix and the accuracy.

Procedure

1. Store the training examples (4 features: Fever, Cough, Headache, BodyPain; class: Flu) and the test examples as lists in Python.
2. Training: count the examples of each class to find the prior probability P(C), and count how often each feature value occurs with each class to find the likelihood P(x | C).
3. Apply Laplace smoothing, P(x | C) = (count + 1) / (class count + number of values of the feature), so that no probability becomes zero.
4. Prediction: for every test example calculate the posterior value P(C) x P(x1 | C) x P(x2 | C) x ... for each class and choose the class with the maximum value.
5. Compare the predicted class with the actual class of every test example and build the confusion matrix (rows = actual class, columns = predicted class).
6. Calculate the accuracy = (sum of the diagonal of the confusion matrix / total test examples) x 100 and display the results.

Coding (Python)

```
# Naive Bayes Classifier - Flu Diagnosis Data Set
from collections import Counter

features = ['Fever', 'Cough', 'Headache', 'BodyPain']

# Training data: [Fever, Cough, Headache, BodyPain, Class]
train = [
    ['Yes', 'Yes', 'Yes', 'Yes', 'Yes'],
    ['Yes', 'Yes', 'No', 'Yes', 'Yes'],
    ['No', 'No', 'No', 'No', 'No'],
    ['No', 'Yes', 'No', 'No', 'No'],
    ['Yes', 'No', 'Yes', 'Yes', 'Yes'],
    ['No', 'No', 'Yes', 'No', 'No'],
    ['Yes', 'Yes', 'Yes', 'No', 'Yes'],
    ['No', 'Yes', 'Yes', 'No', 'No'],
    ['Yes', 'No', 'No', 'Yes', 'Yes'],
    ['No', 'No', 'No', 'Yes', 'No'],
    ['Yes', 'Yes', 'No', 'No', 'No'],
    ['No', 'Yes', 'Yes', 'Yes', 'Yes'],
]

# Test data: [Fever, Cough, Headache, BodyPain, Actual class]
test = [
    ['Yes', 'Yes', 'Yes', 'Yes', 'Yes'],
    ['No', 'No', 'No', 'No', 'No'],
    ['Yes', 'No', 'No', 'No', 'No'],
    ['No', 'Yes', 'No', 'Yes', 'Yes'],
    ['Yes', 'Yes', 'No', 'Yes', 'Yes'],
```

```
    ['No', 'Yes', 'No', 'No', 'No'],
]

def train_naive_bayes(train):
    labels = [r[-1] for r in train]
    class_count = Counter(labels)
    n_features = len(train[0]) - 1
    # values[i] = set of all values taken by feature i
    values = [set(r[i] for r in train) for i in range(n_features)]
    # count[(class, i, value)] = number of training rows
    count = Counter()
    for r in train:
        for i in range(n_features):
            count[(r[-1], i, r[i])] += 1
    return class_count, values, count

def predict(model, x, n_rows):
    class_count, values, count = model
    best_class, best_prob = None, -1
    for c in class_count:
        prob = class_count[c] / n_rows                    # prior P(c)
        for i, v in enumerate(x):
            # likelihood P(v | c) with Laplace smoothing
            prob *= (count[(c, i, v)] + 1) / (class_count[c] + len(values[i]))
        if prob > best_prob:
            best_class, best_prob = c, prob
    return best_class

model = train_naive_bayes(train)
classes = sorted(model[0])

actual, predicted = [], []
print("Test sample | Actual | Predicted")
for row in test:
    p = predict(model, row[:-1], len(train))
    actual.append(row[-1])
    predicted.append(p)
    print(row[:-1], "|", row[-1], "|", p)

# Confusion matrix (rows = actual class, columns = predicted class)
matrix = [[0] * len(classes) for _ in classes]
for a, p in zip(actual, predicted):
    matrix[classes.index(a)][classes.index(p)] += 1

print("\nConfusion Matrix")
print("Class order (rows = actual, columns = predicted):", classes)
for c, row in zip(classes, matrix):
    print("Actual", c, ":", row)

correct = sum(matrix[i][i] for i in range(len(classes)))
accuracy = correct / len(test) * 100
print("\nCorrect predictions :", correct, "out of", len(test))
print("Accuracy :", round(accuracy, 2), "%")
```

Input

Training data set (12 examples):

Fever	Cough	Headache	BodyPain	Flu
Yes	Yes	Yes	Yes	Yes
Yes	Yes	No	Yes	Yes

Fever	Cough	Headache	BodyPain	Flu
No	No	No	No	No
No	Yes	No	No	No
Yes	No	Yes	Yes	Yes
No	No	Yes	No	No
Yes	Yes	Yes	No	Yes
No	Yes	Yes	No	No
Yes	No	No	Yes	Yes
No	No	No	Yes	No
Yes	Yes	No	No	No
No	Yes	Yes	Yes	Yes

Test data set (6 examples):

Fever	Cough	Headache	BodyPain	Flu
Yes	Yes	Yes	Yes	Yes
No	No	No	No	No
Yes	No	No	No	No
No	Yes	No	Yes	Yes
Yes	Yes	No	Yes	Yes
No	Yes	No	No	No

Output

```
Test sample | Actual | Predicted
['Yes', 'Yes', 'Yes', 'Yes'] | Yes | Yes
['No', 'No', 'No', 'No'] | No | No
['Yes', 'No', 'No', 'No'] | No | No
['No', 'Yes', 'No', 'Yes'] | Yes | No
['Yes', 'Yes', 'No', 'Yes'] | Yes | Yes
['No', 'Yes', 'No', 'No'] | No | No

Confusion Matrix
Class order (rows = actual, columns = predicted): ['No', 'Yes']
Actual No : [3, 0]
Actual Yes : [1, 2]

Correct predictions : 5 out of 6
Accuracy : 83.33 %
```

Result

Thus the Naïve Bayes algorithm was implemented in Python successfully. The classifier predicted **5 out of 6** test examples correctly, as shown in the confusion matrix, and achieved an accuracy of **83.33 %**.